

Deliverable 1: Project Proposal & Requirements Specification

By Ryan Beikrasouli

1. Select your game option and provide a brief justification:

The game option I ended up selecting was to create a tower defence game. The way I was able to narrow down my options was as simple. Firstly, Mr Sullivan informed me that he would like me to create a game different to my last year one, so that ruled out the space shooter option. Next, I thought about what games I like to play. This led me to the conclusion that out of all the remaining game options I have spent most of my time playing and even watching people play tower defence games which means I have the greatest understanding of the concept of a tower defence game and what a basic tower defence game should include thus making it the best option for my game. And in complete honesty I just find playing tower defence games to be more relaxing and fun than the other game options.

2. Outline your requirements:

a. Problem definition:

I intend to create a simple but fun tower defence game. The objective of the game would be to survive all the levels by protecting your base and killing the waves of monsters using placeable and upgradeable turrets. As with all games the higher the levels progress the more challenging they become with harder maps (e.g. with shorter paths that the monsters need to travel) and with more powerful monsters (with higher HP and possibly bosses/mini bosses with tower breaking or freezing abilities depending how much time I find I have). My game features will be made achievable through the use of modularisation and object oriented programming practices.

b. Target Audience:

My target audience mainly consists of the current year 12 cohort. My reason for this is due to the main people who are going to be testing and playing my game (besides Mr Sullivan) are going to be from the year 12 cohort (my year cohort). Another reason for selecting the year 12 cohort as my target audience is that tower defence games generally tend to have a higher player base percentage aged between teenager to young adults so it would be reasonable to ensure that player base enjoys my created game.

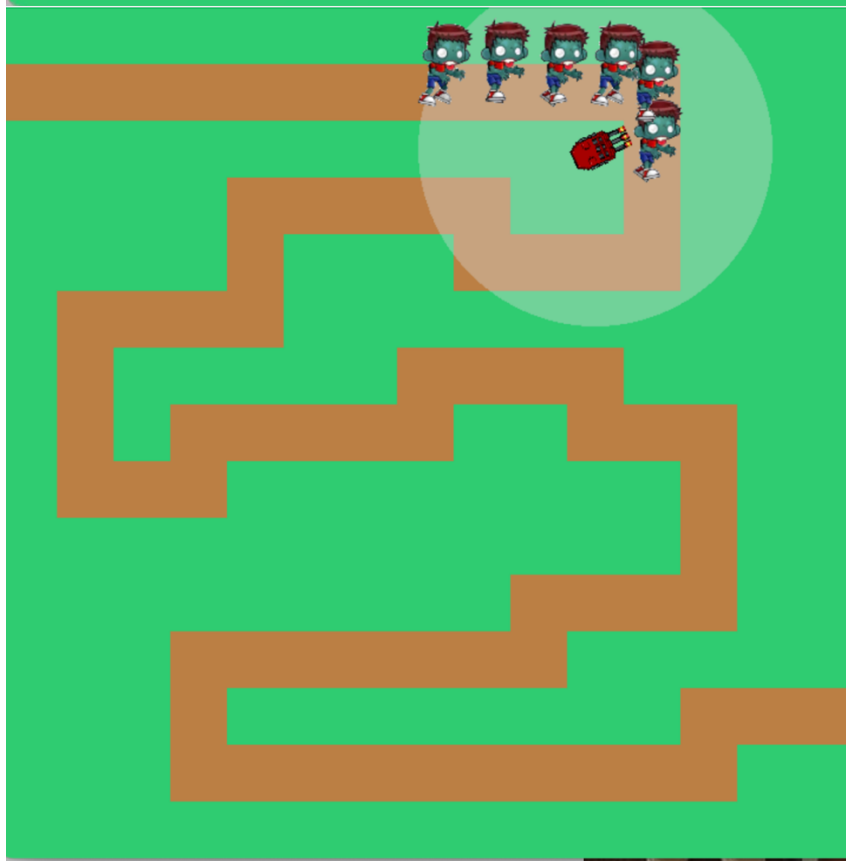
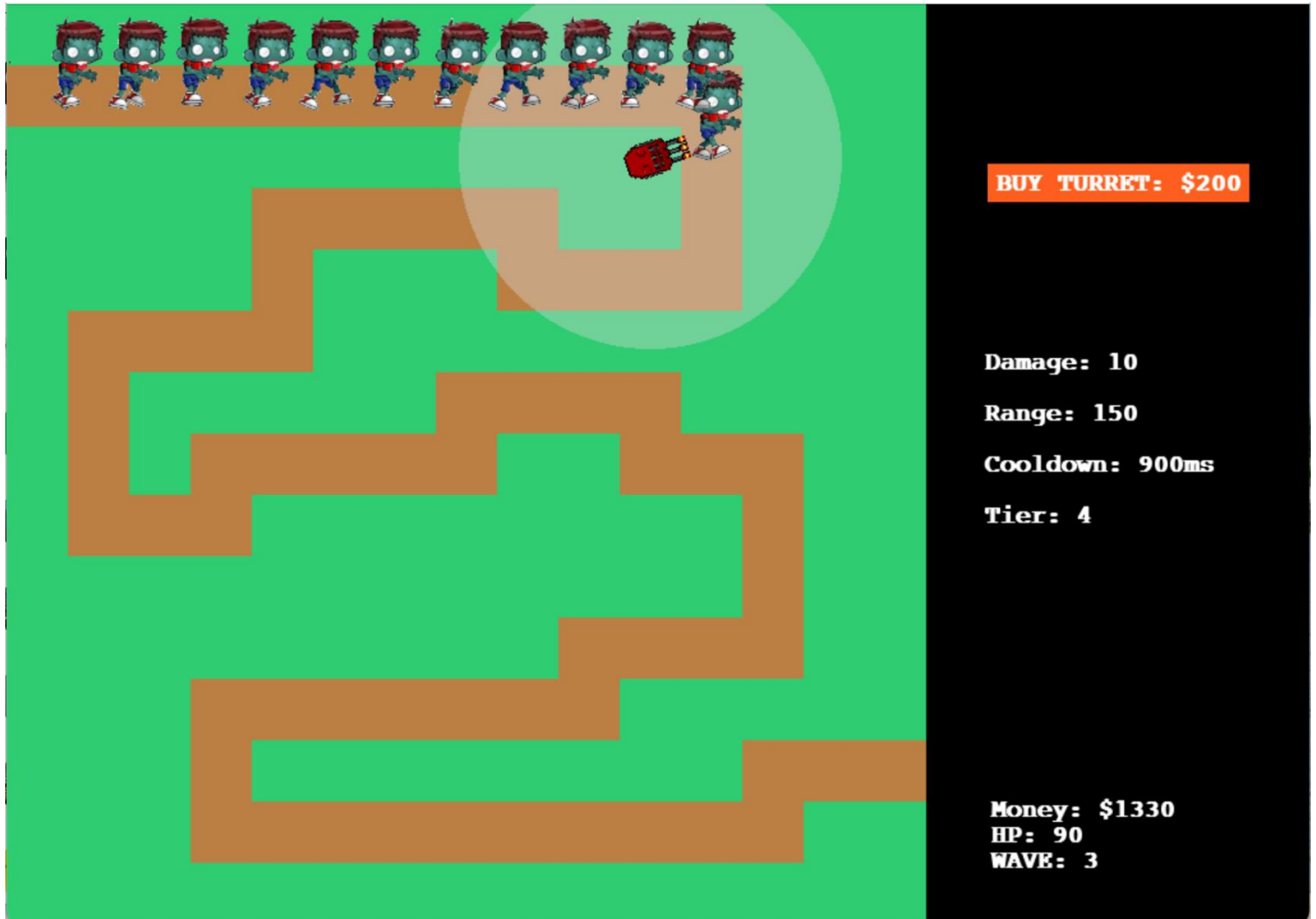
c. Objectives:

For my game to be deemed a success the objectives/requirements that it must achieve are as follows:

1. There must be multiple maps with a path which monsters can follow
2. I must create a method for the monsters which would allow them to navigate the maps I make without having to give the exact x/y coordinates every time. This is achievable through the use of waypoints added to the map thanks to the Tiled application which I decided to use to help assist me with the map creation.
3. Monsters must spawn in waves of increasing difficulty until the final wave
4. If monsters reach the end of the path the player's base must take damage
5. There must be multiple unique monsters with unique attributes such as different speeds and health levels (being balanced of course to ensure the game is still possible even if difficult to beat)
6. The player must be able to buy and place down turrets only on the grassy areas
7. The player must be able to upgrade turrets which would boost their stats such as damage, range and attack speed.
8. Killing monsters and finishing waves must give the player some income which can be used to buy or upgrade their turrets
9. There must be a variety of turrets with different upgrades and attributes such as range, damage and attack speed
10. There must be multiple levels with increasing difficulty each with different maps, new enemies and new unlocked turrets for the player to use
11. The player must be given a score after each level based on their performance (i.e. how fast they complete the level and how much damage they took)

3. Initial designs:

a. Design Sketch:



b. Class Diagrams:

World()

```
+ Wave: int
+ Health: int
+ Money: int
+ enemyList: list
- worldData: dict
- image: pygame.surface.Surface
+ waypoints: list
+ tilemap: list
+ spawnedEnemies: int
+ killedEnemies: int
+ missedEnemies: int

+ processData()
- processWaypoints(data:dict)
+ processEnemies()
+ checkWaveFinished()
+ prepareNewWave()
+ draw(surface: pygame.surface.Surface)
```

Button(pygame.sprite.Sprite)

```
- X: int
- Y: int
- Height: int
- Width: int
- squareCheck: bool
- font: pygame.font.Font
- clicked: bool
- singleClick: bool
- textColour: Tuple[int, int, int]
+ active: bool
- Square: pygame.Rect
- SquareOnColour: Tuple[int, int, int]
- SquareOffColour: Tuple[int, int, int]
- text: str
- textRect: pygame.Rect
- text_on: pygame.surface.Surface
- text_off: pygame.surface.Surface

+ MouseCheck(SCREEN,
disabled:bool=False,
disabledColour=None, newText=None)
+ MouseClick()
```

Turret(pygame.sprite.Sprite)

```
+ tileX: int
+ tileY: int
+ x: int
+ y: int
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ tier: int
+ range: int
+ cooldown: int
+ damage: int
- lastShot: int

- updateTime: int
+ selected: bool
+ target: Enemy
- doubleSpeed: bool
- upgradeSfx: pygame.mixer.Sound
+ allTurretSheets: list
+ animationImages: list
+ imageIndex: int
+ rotationAngle: float | int
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ Rect pygame.Rect
- rangeImage: pygame.surface.Surface
- rangeRect: pygame.Rect

+ setUpRange()
+ update(allEnemiesGroup: pygames.sprite.Group, doubleSpeed:bool)
- selectTarget(allEnemiesGroup: pygames.sprite.Group)
+ loadImages(spoteSheet: pygame.surface.Surface)
- updateAnimation()
+ upgrade()
+ draw(surface: pygame.surface.Surface)
```

Enemy(pygame.sprite.Sprite)

```
+ health: int
+ worth: int
+ damage: int
+ movementSpeed: int
- speedDoubled: bool
- waypoints: list
+ pos: Vector2 (similar to Tuple[int, int] | Tuple[float, float])
- nextWaypointIndex: int
+ loops: int
- rotationAngle: float
+ imageIndex: int
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
- movement: Vector2 (similar to Tuple[int, int] | Tuple[float, float])
+ update(world: World, doubleSpeed: bool)
- move(world: World)
- updateRotation()
+ animate()
- checkAlive(world: World)
```

Zombie(Enemy)

```
+ health: int
+ movementSpeed: int
+ worth: int
+ damage: int
+ OriginalImage: pygame.surface.Surface
+ animate()
```

Skeleton(Enemy)

```
+ health: int
+ movementSpeed: int
+ worth: int
+ damage: int
+ OriginalImage: pygame.surface.Surface
+ animate()
```

Cannon(Turret)

```
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ range: int
+ cooldown: int
+ damage: int
+ allTurretSheets: list
+ animationImages: list
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
```

Machinelaser(Turret)

```
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ range: int
+ cooldown: int
+ damage: int
+ allTurretSheets: list
+ animationImages: list
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
+ allTurretSheets: list
```

4. Pseudocode example:

```
FUNCTION createTurret(mousePosition allTurretsGroup, world, turretType)
    tileX = mousePosition[0] // TILE_SIZE
    tileY = mousePosition[1] // TILE_SIZE
    mouseTileNum = tileY * TILE_ROWS + tileX
    IF world.tilemap[mouseTileNum] IN GRASS_TILE_VALUES THEN
        tileFree = True

        FOR turret IN allTurretsGroup DO
            IF (tileX, tileY) == (turret.tileX, turret.tileY) THEN
                tileFree = False
            END IF
        END FOR

        IF tileFree THEN
            IF turretType == "cannon" THEN
                newTurret = Cannon(tileX, tileY)
            ELSE IF turretType == "machinelaser" THEN
                newTurret = Machinelaser(tileX, tileY)
            END IF

            IF world.money - newTurret.cost >= 0 THEN
                allTurretsGroup.add(newTurret)
                world.money -= newTurret.cost
            END IF
        END IF

    END IF

    return allTurretsGroup

END createTurret
```